



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe Avance 7 - Problema 2

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

22 de junio 2026

1 El método de cuadratura Gaussiana

La **cuadratura Gaussiana** es un método de integración numérica que aproxima una integral definida mediante una suma ponderada de evaluaciones de la función:

$$\int_a^b f(t) dt \approx \sum_{i=1}^k w_i f(t_i).$$

A diferencia de las reglas de Newton–Cotes (trapecio, Simpson), que utilizan nodos t_i *equiespaciados*, la cuadratura Gaussiana elige tanto los **nodos** x_i como los **pesos** w_i de manera óptima. Esta libertad adicional permite que una regla con k nodos integre de forma **exacta** cualquier polinomio de grado menor o igual a $2k - 1$. Por ello, para un mismo número de evaluaciones de f , la cuadratura Gaussiana suele ser considerablemente más precisa que los métodos de nodos equiespaciados.

Los nodos x_i son las raíces del polinomio de Legendre de grado k y los pesos w_i se obtienen a partir de ellos. En este trabajo se utilizan los valores tabulados para $k = 3$.

2 Fórmula en el intervalo $[-1, 1]$

La cuadratura Gaussiana está definida de forma canónica sobre el intervalo de referencia $[-1, 1]$:

$$\boxed{\int_{-1}^1 f(x) dx \approx \sum_{i=1}^k w_i f(x_i)}$$

Para $k = 3$ nodos, los valores tabulados de nodos y pesos son:

$$\begin{aligned} x_1 &= -\sqrt{\frac{3}{5}}, & x_2 &= 0, & x_3 &= \sqrt{\frac{3}{5}}, \\ w_1 &= \frac{5}{9}, & w_2 &= \frac{8}{9}, & w_3 &= \frac{5}{9}. \end{aligned}$$

Esta regla de 3 nodos integra de forma exacta todo polinomio de grado $\leq 2(3) - 1 = 5$.

3 Cambio de variable hacia un intervalo arbitrario $[a, b]$

La fórmula anterior solo sirve para $[-1, 1]$. Para aproximar

$$\int_a^b f(t) dt$$

sobre un intervalo arbitrario $[a, b]$, se necesita una transformación lineal $x \mapsto t$ que lleve el intervalo de referencia $[-1, 1]$ al intervalo $[a, b]$. Usamos t para la variable original (para no confundirla con la variable nueva x). Queremos una transformación que cumpla:

$$x = -1 \longrightarrow t = a, \quad x = 1 \longrightarrow t = b.$$

La transformación lineal que logra esto es:

$$t = \frac{b-a}{2}x + \frac{b+a}{2} \quad \Longleftrightarrow \quad t = \frac{(b-a)x + (b+a)}{2}.$$

Verificación de los extremos

Si $x = -1$:

$$t = \frac{b-a}{2}(-1) + \frac{b+a}{2} = \frac{-b+a+b+a}{2} = \frac{2a}{2} = a.$$

Si $x = 1$:

$$t = \frac{b-a}{2}(1) + \frac{b+a}{2} = \frac{b-a+b+a}{2} = \frac{2b}{2} = b.$$

Entonces la transformación cambia el intervalo: $[-1, 1] \rightarrow [a, b]$.

Transformación del diferencial

Falta transformar el diferencial. Derivando t respecto a x :

$$\frac{dt}{dx} = \frac{b-a}{2} \implies dt = \frac{b-a}{2} dx.$$

Integral resultante

Sustituyendo el cambio de variable y el diferencial, la integral original se convierte en:

$$\int_a^b f(t) dt = \int_{-1}^1 f\left(\frac{(b-a)x + (b+a)}{2}\right) \frac{b-a}{2} dx.$$

Definiendo

$$g(x) = \frac{b-a}{2} f\left(\frac{(b-a)x + (b+a)}{2}\right),$$

queda finalmente

$$\int_a^b f(t) dt = \int_{-1}^1 g(x) dx,$$

de modo que aplicando la regla de $[-1, 1]$ obtenemos la aproximación en $[a, b]$:

$$\boxed{\int_a^b f(t) dt \approx \frac{b-a}{2} \sum_{i=1}^k w_i f\left(\frac{(b-a)x_i + (b+a)}{2}\right)}.$$

4 Versión simple, compuesta e iterativa

Simple

La versión **simple** aplica la cuadratura Gaussiana de k nodos *una sola vez* sobre todo el intervalo $[a, b]$. Es rápida y muy precisa cuando la función es suave y el intervalo no es muy amplio, pero su exactitud queda limitada por el número fijo de nodos.

Compuesta

La versión **compuesta** divide $[a, b]$ en n subintervalos de igual longitud $h = (b-a)/n$ y aplica la cuadratura Gaussiana simple en cada uno, sumando los resultados:

$$\int_a^b f(t) dt \approx \sum_{j=1}^n \int_{a_j}^{b_j} f(t) dt, \quad a_j = a + (j-1)h, \quad b_j = a + jh.$$

Al usar nodos sobre subintervalos más pequeños, el error disminuye al aumentar n , lo que la hace más precisa que la versión simple a costa de más evaluaciones de la función.

Iterativa

La versión **iterativa** automatiza la elección de n . Parte de $n = 1$ y *duplica* el número de subintervalos en cada paso, comparando dos aproximaciones consecutivas. El proceso se detiene cuando la diferencia entre ellas es menor que una tolerancia prefijada:

$$\text{err} = |I_n - I_{n/2}| < \text{tol},$$

o cuando se alcanza un número máximo de subintervalos. Así se obtiene una aproximación con una precisión controlada sin tener que fijar n a mano.

5 Comandos y funciones de GNU Octave

El programa se organiza en una función principal `p2_av7` y tres subfunciones. Los comandos y construcciones principales utilizados son:

- **Funciones anónimas** `@(x) ...`: definen la función a integrar, por ejemplo `f = @(x) exp(-x.^2)`.
- `nodos_pesos(k)`: función auxiliar que devuelve los vectores de nodos `x` y pesos `w` tabulados para k nodos.
- **Operaciones vectorizadas** `(.*, ./)`: aplican el cambio de variable a todo el vector de nodos de una sola vez.
- `arrayfun(f, t)`: evalúa la función `f` sobre cada elemento del vector `t`.
- `sum(...)`: realiza la suma ponderada $\sum_i w_i f(t_i)$.
- **Ciclo for ... endfor**: recorre los n subintervalos en la versión compuesta.
- **Ciclo while ... endwhile** con `return`: controla la duplicación de subintervalos en la versión iterativa hasta cumplir la tolerancia.
- `abs(...)`: calcula el error como valor absoluto de la diferencia entre iteraciones.
- `warning(...)`: avisa si se alcanza el máximo de subintervalos sin cumplir la tolerancia.

A continuación se muestran las tres subfunciones principales:

Listing 1: Cuadratura Gaussiana simple

```
1 function Is = cuad_gauss(f, a, b, k)
2   [x, w] = nodos_pesos(k);
3   t = ((b-a).*x + (b+a)) ./ 2;
4   Is = ((b-a)/2) * sum(w .* arrayfun(f, t));
5 endfunction
```

Listing 2: Cuadratura Gaussiana compuesta

```
1 function Ic = cuad_gauss_comp(f, a, b, k, n)
2   h = (b-a)/n;
3   Ic = 0;
4   for j = 1:n
5       aj = a + (j-1)*h;
6       bj = a + j*h;
7       Ic = Ic + cuad_gauss(f, aj, bj, k);
8   endfor
9 endfunction
```

Listing 3: Cuadratura Gaussiana iterativa

```

1 function [I, err, n] = cuad_gauss_iter(f, a, b, k)
2     tol = 1e-8;
3     nmax = 100e6;
4     n = 1;
5     I_ant = cuad_gauss_comp(f, a, b, k, n);
6     while n < nmax
7         n = 2*n;
8         if n > nmax, n = nmax; endif
9         I = cuad_gauss_comp(f, a, b, k, n);
10        err = abs(I - I_ant);
11        if err < tol, return; endif
12        I_ant = I;
13    endwhile
14    warning("Se alcanzo el maximo de subintervalos sin cumplir la tolerancia.");
15 endfunction

```

““latex

6 Resultados obtenidos

Se evaluó la integral

$$\int_2^5 e^t \cos(t) dt,$$

cuyo valor de referencia se obtuvo mediante la antiderivada exacta

$$\int_2^5 e^t \cos(t) dt = \left[\frac{e^t(\sin(t) + \cos(t))}{2} \right]_2^5 \approx -51.930848628576867707.$$

Se usaron los parámetros $k = 7$ nodos y, para la versión compuesta, $n = 20$ subintervalos. Los resultados obtenidos fueron:

Método	Aproximación	Error absoluto	Parámetros
Simple	-51.93084862693622	1.64×10^{-9}	$k = 7$
Compuesta	-51.93084862857686	7.71×10^{-15}	$k = 7, n = 20$
Iterativa	-51.93084862857665	2.18×10^{-13}	$k = 7$
Exacto	-51.930848628576867707	—	—

Cuadro 1: Aproximaciones de $\int_2^5 e^t \cos(t) dt$ por los tres métodos.

7 Comparación de las tres aproximaciones

Los resultados muestran que las tres versiones del método entregan una aproximación muy precisa para la integral estudiada. Esto se debe a que la función

$$f(t) = e^t \cos(t)$$

es suave en todo el intervalo $[2, 5]$, es decir, no presenta discontinuidades, singularidades ni cambios bruscos que dificulten la aproximación numérica.

- La **versión simple**, usando una sola cuadratura Gaussiana de orden $k = 7$ sobre todo el intervalo $[2, 5]$, obtuvo

$$I_s \approx -51.93084862693622.$$

Su error absoluto fue aproximadamente

$$1.64 \times 10^{-9}.$$

Aunque es el mayor error entre los tres métodos, sigue siendo un error muy pequeño. Esto muestra que una cuadratura Gaussiana de orden alto puede ser bastante eficiente incluso sin subdividir el intervalo.

- La **versión compuesta**, con $k = 7$ y $n = 20$ subintervalos, obtuvo

$$I_c \approx -51.93084862857686.$$

Su error absoluto fue aproximadamente

$$7.71 \times 10^{-15}.$$

Este resultado es prácticamente igual al valor exacto dentro de la precisión numérica mostrada. La mejora se debe a que el intervalo $[2, 5]$ se divide en subintervalos más pequeños y en cada uno se aplica una cuadratura Gaussiana local.

- La **versión iterativa** obtuvo

$$I \approx -51.93084862857665.$$

En este caso, el algoritmo se detuvo con $n = 2$ subintervalos, ya que la diferencia entre dos aproximaciones consecutivas fue aproximadamente

$$\text{err} \approx 1.64 \times 10^{-9},$$

menor que la tolerancia establecida de 10^{-8} . Su error absoluto respecto al valor de referencia fue aproximadamente

$$2.18 \times 10^{-13}.$$

Esto indica que la función iterativa no necesariamente busca usar una gran cantidad de subintervalos, sino detenerse cuando el cambio entre aproximaciones sucesivas ya es suficientemente pequeño.

En conclusión, para esta función suave la cuadratura Gaussiana converge muy rápidamente. La versión simple ya ofrece una aproximación muy buena con $k = 7$, mientras que la versión compuesta con $n = 20$ alcanza una precisión casi idéntica al valor exacto. Por su parte, la versión iterativa resulta práctica porque ajusta automáticamente el número de subintervalos y evita elegir n por ensayo y error. En este ejemplo, solo necesitó $n = 2$ subintervalos para cumplir la tolerancia solicitada.